

Introduccion a POO

Juan Ignacio Raggio

May 3, 2025

Contents

1	DOUBTS DUDAS poo	2
2	POO	2
3	Instancias	2
3.0.1	Variable de instancia	2
3.0.2	Variable de clase -> STATIC	2
3.1	Relaciones entre instancias EJEMPLOS:INSTANCIAS . . .	3
4	Herencia	3
4.1	Herencia multiple	3
4.2	Ejemplos de herencia:	3
4.2.1	Ejemplos CORRECTOS	4
4.2.2	Ejemplos INCORRECTOS	4
4.3	Consejos para diseñar herencia	4
5	Polimorfismo	5
5.1	Tres tipos de polimorfismos:	6
5.1.1	Sobrecarga: Distintas clases con metodos con el mismo nombre SOBRECARGA:OVERLOADING	6
5.1.2	Parametrico: Dentro de una misma clase, metodos con el mismo nombre pero distintos parametros (ya sean cantidad o tipo) PARAMETRICPOLYMORPHISM	6
5.1.3	Redefinicion: Una clase hija sobreescibe un metodo de una clase padre SOBREENSCRIPCION:OVERRIDING	6
6	Metodo y mensaje	6
6.1	Metodo de clase -> STATIC	6
6.2	Metodo de instancia	7

1 DOUBTS DUDAS poo

2 POO

- Sub-paradigma del paradigma imperativo
- Un programa es un conjunto de objetos que interactúan a través de mensajes
- Los **objetos** son **instancias** de una **clase**
- Clase -> **Encapsula** Propiedades y métodos
- Estado de un objeto -> Valores de los atributos
- El estado de un objeto se modifica a través de métodos

3 Instancias

- Una instancia es una ocurrencia de una determinada clase

```
Date d1 = new Date(2020, 6, 20);  
Date d2 = d1;
```

En este caso d1 y d2 son dos variables distintas pero referencian a la misma clase

3.0.1 Variable de instancia

- Variable que se relaciona con una **única instancia de una clase** -> Son independientes de las distintas instancias, cada instancia tiene sus propios valores de variables de instancia

```
Date d1 = new Date(2020, 6, 20);  
Date d2 = new Date(2020, 9, 23);
```

En este caso, d1.year = d2.year Pero d1.month != d2.month

3.0.2 Variable de clase -> STATIC

- Estas variables son también llamadas estáticas y son compartidas por todas las instancias de un objeto. Es una **especie de variable global** para clases
- No es necesario que exista una instancia para que exista la variable

3.1 Relaciones entre instancias EJEMPLOS:INSTANCIAS

- La herencia crea una relacion "es una" entre clases
- La composicion crea una relacion de dependencia "tiene un" entre instancias (hay una relacion de "una es dueña de la otra"). En composicion si muere el dueño, muere el objeto asociado con la relacion de tenencia. Cuando tenemos una composicion quiere decir que tenemos un atributo del tipo de la clase poseida
 - La clase persona tiene una o mas tarjetas de credito
 - La clase remito tiene una lista de items
 - Un pais tiene un conjunto de ciudades
- La asociacion son dos objetos trabajando juntos
 - Un alumno tiene asociado un plan, un conjunto de materias aprobadas y un conjunto de cursos en los que esta inscripto
 - Un curso tiene una lista de alumnos activos

MUY IMPORTANTE para una relacion de jerarquia: Es importantisimo que vamos a crear una relacion solo si tiene sentido el **COMPORTAMIENTO** de las clases, si lo que se comparten son solo atributos, no vamos a crear una relacion de jerarquia

4 Herencia

- Es definir una nueva clase en base a una clase existente
- La subclase hereda los atributos y metodos de la clase "padre"
- Se puede decir que una subclase **extiende** o **hereda** a la clase padre

4.1 Herencia multiple

Algunos lenguajes permiten la herencia multiple Java y Ruby no lo permiten

4.2 Ejemplos de herencia:

Suponiendo que DateTime extiende a Date para que tenga no solo YYYY/MM/DD sino que tambien tenga la hora y algun otro indicador de tiempo

```
Date d1;  
DateTime d2 = new DateTime(...);
```

4.2.1 Ejemplos CORRECTOS

```
d1 = new DateTime(...);  
d1 = new Date(...);  
d1 = d2; // Es correcto ya que ignora la extension ( entiendo que copia los primeros x
```

4.2.2 Ejemplos INCORRECTOS

```
d2 = d1; // Pues d2 necesita mas informacion que la que le puede brindar d1 y copiaria  
d2 = new Date(...);
```

4.3 Consejos para diseñar herencia

1. Agrupar operaciones y propiedades en comun en superclases
2. No usar campos "protegidos"
 - Los campos protegidos son accesibles por los metodos de instancia de la clase y las clases que la extienden. Por lo que la clase **no puede controlar el estado de los campos "protegidos"**, pueden ser modificados sin problema
 - La solucion es usar getters y setters y agregar la keyword **protected** a los metodos
3. Usar herencia para modelar la relacion "**es-un**"
 - Un cuadrado es un rectangulo
 - Un rectangulo es una figura
 - Un triangulo es una figura
 - Tren, Subte, Colectivo, . . . , son medios de transporte
 - Un gato, un perro son animales
 - Un estudiante, es una persona?
4. Usar herencia si **TODOS** los metodos heredados tienen sentido
 - A veces es conveniente usar clases inmutables si no queremos que todos los metodos se usen
5. Usar **polimorfismo**, NO usar informacion de tipos Dada la siguiente clase:

- ☒ NO HACER (tener un metodo method() distinto para cada clase) -> En un examen esto te puede anular el ejercicio

```
if (a instanceof Class1)
    a.method1();
else if (a instanceof Class2)
    a.method2();
```

- ☐ Si hacer (Usando polimorfismo)

```
class MyClass {
    void method() {};
}
class Class1 extends MyClass {
    @Override
    void method() {};
}
class Class2 extends MyClass {
    @Override
    void method() {};
}

a.method();
```

5 Polimorfismo

El Polimorfismo es muy importante de usar para no tener que estar validando parametros todo el tiempo o tener que crear metodos que me confirmen si un campo es x, que haga tal cosa pero si es y, que haga otra y quiere decir que un metodo **Posee varias formas diferentes** Esta bueno usar polimorfismo tambien en los constructores si es necesario

5.1 Tres tipos de polimorfismos:

5.1.1 Sobrecarga: Distintas clases con metodos con el mismo nombre SOBRECARGA:OVERLOADING

5.1.2 Parametrico: Dentro de una misma clase, metodos con el mismo nombre pero distintos parametros (ya sean cantidad o tipo) PARAMETRICPOLYMORPHISM

5.1.3 Redefinicion: Una clase hija sobreescribe un metodo de una clase padre SOBREESCRIPCION:OVERRIDING

```
public class GF {  
    public void method() {  
        System.out.println("This is a method in GF");  
    }  
}  
public class S extends GF
```

6 Metodo y mensaje

- Un mensaje es enviado por un objeto para invocar comportamiento
- Un metodo se implementa en una clase, y determina como debe actuar el objeto cuando recibe un mensaje
- Los objetos interactuan enviandose mensajes unos a otro. Tras la recepcion de un mensaje el objeto actuara
- Enviar un mensaje es invocar a un metodo de instancia

6.1 Metodo de clase -> STATIC

Pueden ser invocados sin que exista una instancia (tal como las variables de clase)

```
public class Date {  
    private static String format;  
    private int year, month, day;  
    public static String getFormat() {  
        return format;  
    }  
    public static Date today() {
```

```

        return ...;
    }
}

public class DateTime extends Date {
    private int hour, minute, second;
    ...;
    public boolean isMorning() {
        return hour < 12;
    }
    public static DateTime now() {
        return ...;
    }
}

```

6.2 Metodo de instancia

- Son invocados sobre una instancia de clase especifica
- Pueden alterar el estado de la instancia o retornar algun valor en base al estado actual de la misma